

Production-Ready AI Agents

Security • Evaluation • Observability • Cost • Deployment

DECK 6

From a working POC to an enterprise-ready AI system





From POC to Production

A working agent is not automatically a production system

A POC Demonstrates

- The agent can reason
- Tools can be called
- Workflows can execute
- An LLM can generate useful results

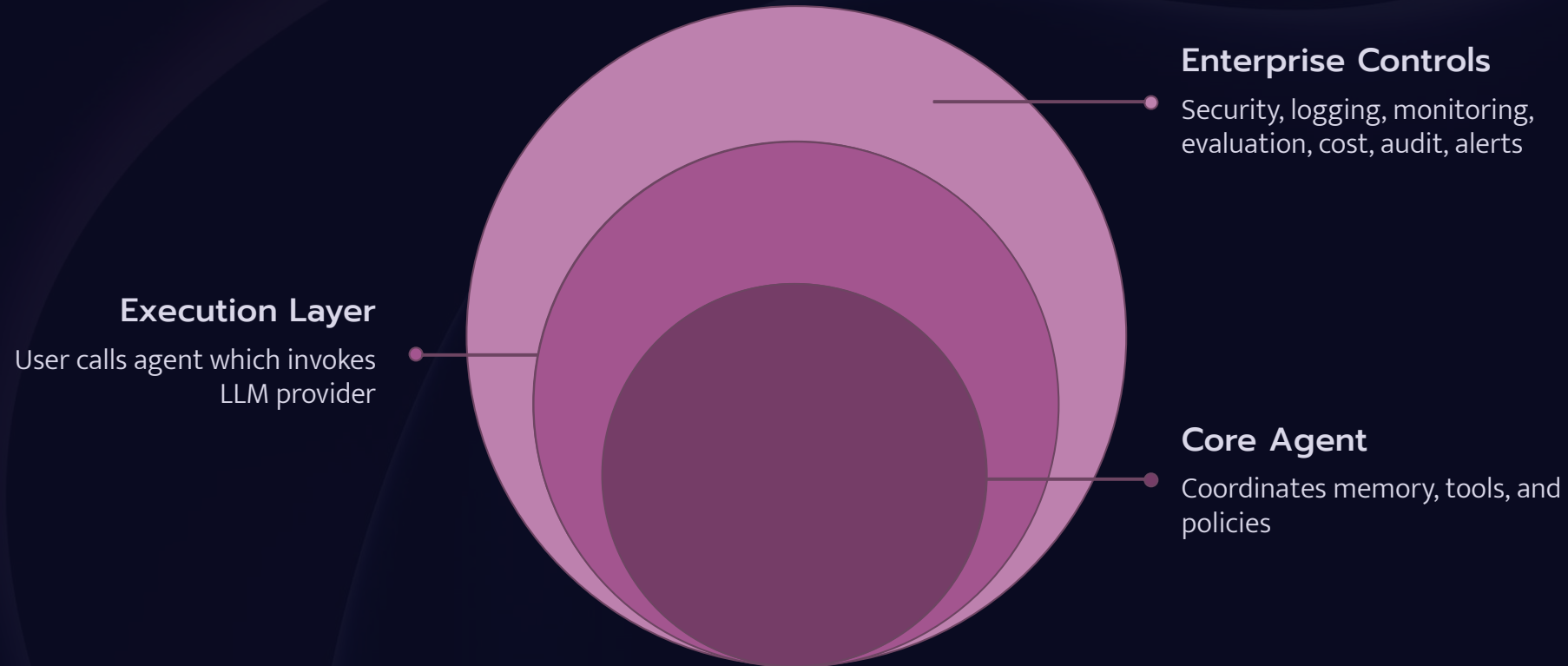
Production Additionally Requires

- Security & authorization
- Reliability & failure recovery
- Observability & tracing
- Evaluation & testing
- Cost control & governance
- Deployment & operational support

"Can the agent work?" must evolve into "**Can the organization safely operate the agent?**" — that shift in question defines the path to production.

Production Agent Architecture

Production is an architecture, not just an `agent.py` file



Every production agent sits within a layered system. The agent itself handles reasoning, but enterprise controls — security, observability, evaluation, and cost — operate as a horizontal layer spanning the entire architecture. No single component is sufficient on its own.

Configuration Management

Never hard-code operational configuration

What Belongs in Configuration

- LLM provider & model selection
- Temperature & max output tokens
- Timeout & retry settings
- API endpoints
- Feature flags
- Environment settings

The Core Principle

The same agent code must be deployable across development, staging, and production environments through configuration changes alone — never through source-code modifications.

Separating code from configuration reduces risk, enables rapid environment switching, and supports safe rollbacks.



Hard-coded operational values are a deployment risk. Externalize everything that varies by environment.

Prompt Versioning

Prompts are part of the application — treat them like code

Prompts in an Enterprise Agent

- System instructions
- Planning prompts
- Tool-selection prompts
- Reviewer prompts
- Output-format instructions

Each of these can independently alter agent behavior. A small wording change can shift planning strategy, tool preference, or output structure in ways that are difficult to detect without versioning.

Track for Every Prompt Version

Version & Date

Sequential identifier and timestamp

Purpose

Why the change was made

Changes

What specifically was modified

Evaluation

Results against test dataset



Secrets and Credentials

Never put credentials inside the agent

Never Store API Keys In

- Python source files or notebooks
- Configuration files or README files
- Git repositories (even private ones)
- Application logs
- `.env` files committed to source control

Lab Standard

API credentials come exclusively from the operating-system environment. The training framework explicitly prohibits `.env` files and credentials embedded in source or configuration. All secrets are delivered through environment variables at runtime — never baked into the codebase.

Authentication vs. Authorization

These are fundamentally different controls — both are required

Authentication

Who are you? Verifies the identity of the caller — user, service, or agent — before any action is considered. Establishes a trusted identity in the system.



Authorization

What are you allowed to do? Enforces boundaries on what an authenticated identity may access or execute. Must be deterministic and enforced by application controls — not the LLM.

 The LLM must never be the authorization decision point. Probabilistic models cannot reliably enforce deterministic access policies. Authorization belongs in code.

Tool Security

Tools are the agent's action boundary — and a critical security surface

A Tool May

- Read a database or call an API
- Create a ticket or send an email
- Update a record or trigger a business process

Each action carries real-world consequence. Tool access must be controlled independently of the LLM. The agent's reasoning may be correct, but the tool layer is where policy is enforced.

Control at the Tool Boundary

Every tool invocation should pass through an authorization check before execution. The check is deterministic: allow or deny based on the caller's identity, the requested action, and the resource scope.

The result of that check — not the LLM's confidence — determines whether the tool runs.

Rate Limiting and Retries

LLM and enterprise APIs will fail — design for it

Common Failure Causes

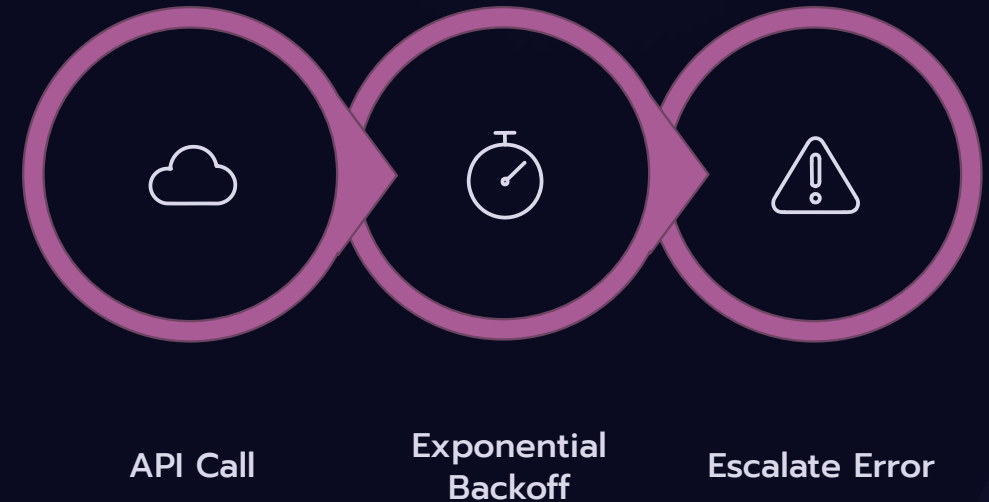
- Provider rate limits
- Temporary network errors
- Service unavailability
- Timeout exceeded
- Provider-side errors

What to Avoid

- Infinite retry loops
- Immediate repeated requests (hammering)
- Retrying non-recoverable errors

Use exponential backoff with jitter between retries, cap the number of attempts, distinguish transient from permanent failures, and define a clear escalation path when all retries are exhausted.

Retry Intelligently



Caching and Cost Control

Not every request needs a fresh LLM call

Cache Where Appropriate

- Stable reference information
- Repeated retrieval results
- Expensive deterministic operations
- Reusable intermediate results

Reduce Tokens

- Better, tighter prompts
- Structured outputs
- Smaller context windows
- Summarization of intermediate results
- Right-sizing model selection

The Cost Equation

Total Cost $\approx$ LLM Calls $\times$ Tokens per Call $\times$ Cost per Token + Tool/API Costs + Infrastructure Costs

Every term is a lever. Reduce unnecessary calls, minimize token volume, and select the most cost-efficient model capable of handling each step in the workflow.

Token Optimization

Agentic workflows amplify token consumption across every step

A Single Request May Generate

01

Planning call

02

Tool selection

03

Tool result interpretation

04

Replanning

05

Review call

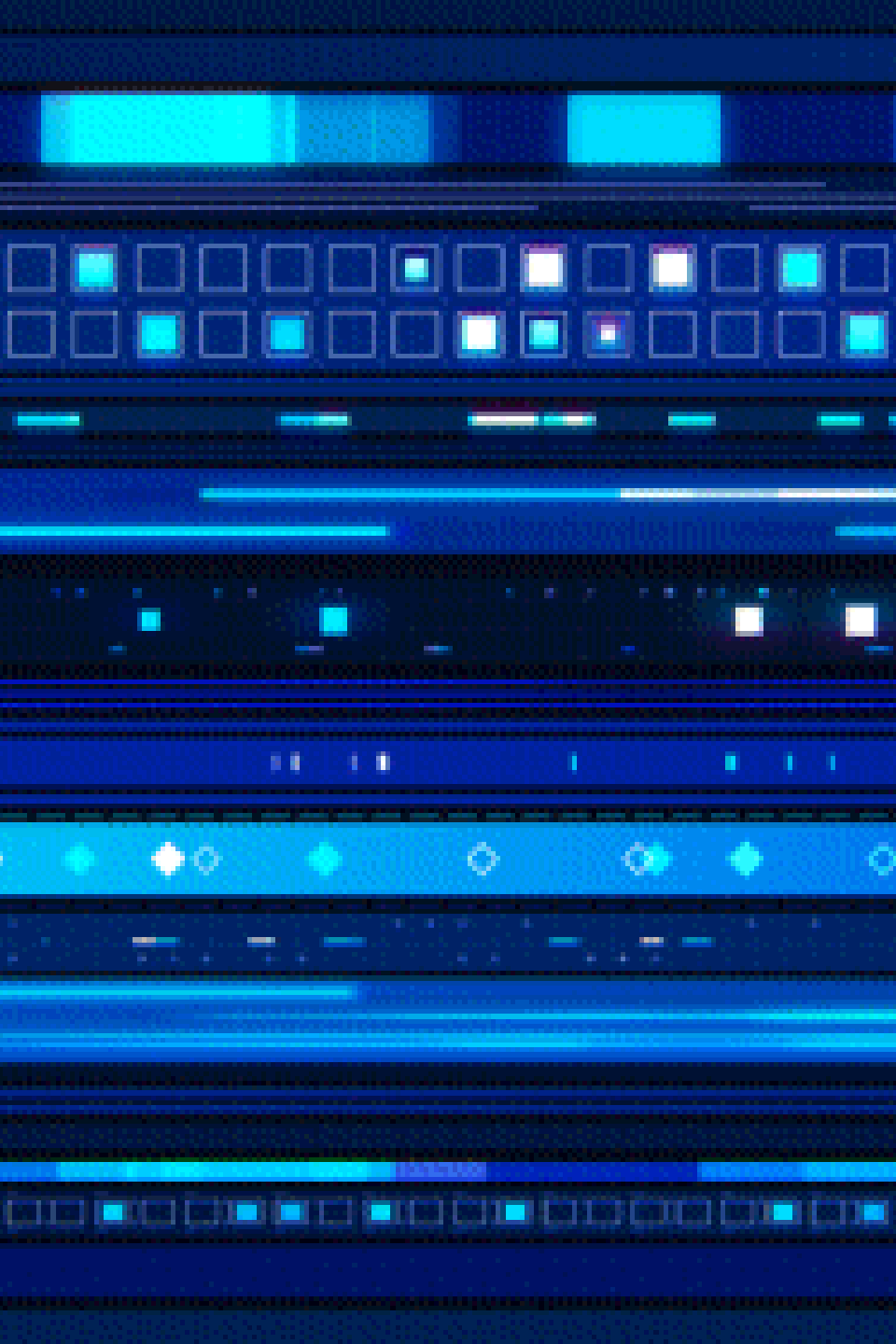
06

Final response generation

Optimization Questions to Ask

- Does this step actually need an LLM?
- Does this step need the full context?
- Can the output be structured to reduce verbosity?
- Can a smaller model handle this task?
- Can intermediate results be summarized?
- Can repeated results be cached?

Optimize the workflow, not just the prompt.



Logging

If you cannot see what the agent did, you cannot reliably operate it.

Log These Events

- Request ID & Agent/session ID
- Workflow state & step transitions
- Tool selected & execution result
- Errors & retry attempts
- Approval events & completion status
- Model/provider & token/cost information

Example Log Sequence

Request: REQ-10245
Agent: PurchaseAgent
Step: policy_check
Tool: get_policy()
Result: SUCCESS
Step: budget_check
Tool: check_budget()
Result: SUCCESS
Review: REQUIRED
Approval: PENDING

Tracing Agent Workflows

Logging vs. Tracing

Logging

Tells you **what** happened — individual events recorded at a point in time.

Tracing

Tells you **how** the workflow happened — the full causal chain across steps, calls, and tools.

What Traces Reveal

- Slow or bottlenecked steps
- Repeated or redundant calls
- Failed tools and retry chains
- Excessive token usage per step
- Unexpected agent behavior

A Traced Workflow

LLM Call #1 — Planning

Tool #1 — Policy API

Tool #2 — Budget API

LLM Call #2 — Review

Human Approval Gate

Monitoring

Production monitoring should answer four fundamental questions

1

Is the system available?

Track errors, timeouts, and service availability. Know when the agent is unreachable before users report it.

2

Is it performing?

Measure latency, completion rate, and tool success rate. Degraded performance is often the first signal of deeper issues.

3

Is it producing useful results?

Monitor evaluation scores, human feedback signals, and task success rates — not just technical uptime.

4

Is it economically viable?

Track token volume, LLM call counts, cost per task, and cost trends. Budgets can erode quickly at scale.

Evaluating AI Agents

Traditional software testing is necessary but not sufficient

Deterministic Code vs. AI Agents

For deterministic code, a fixed input produces a fixed output — testing is straightforward. An AI agent takes the same input and may plan differently, select different tools, and generate varying outputs. No single test case can cover the behavioral space.

Evaluate Multiple Dimensions

- Task success & correctness
- Tool selection & argument validity
- Policy compliance & safety
- Output structure & reliability

What Evaluation Covers

Behavior

Did the agent follow the expected workflow?

Decisions

Were tools chosen and called correctly?

Safety

Were forbidden actions avoided?

Build an Evaluation Dataset

Create representative scenarios before deployment — then run them after every change

Dataset Should Include

- **Normal requests**
The expected happy path
- **Ambiguous & invalid inputs**
Edge cases and malformed requests
- **Tool failures & policy violations**
Failure mode coverage
- **Adversarial cases**
Attempts to bypass controls

For Each Scenario Define

- **Input** — the request as the agent receives it
- **Expected behavior** — the ideal decision path
- **Required tool calls** — which tools must be invoked
- **Forbidden actions** — what the agent must never do
- **Expected output characteristics** — format, content, and constraints

- Run the full dataset after every prompt change, model upgrade, or tool modification.

Testing AI Agents

Test at multiple levels – each level catches different failure modes



Unit Tests

Test individual functions and tools in isolation. Verify that each tool returns correct outputs for known inputs and handles errors gracefully.



Integration Tests

Test the agent together with tools and external systems. Validate that the interfaces, authentication, and data contracts work correctly end-to-end.



Workflow Tests

Test complete agent workflows against realistic multi-step scenarios. Confirm that planning, tool selection, and completion behave as designed.



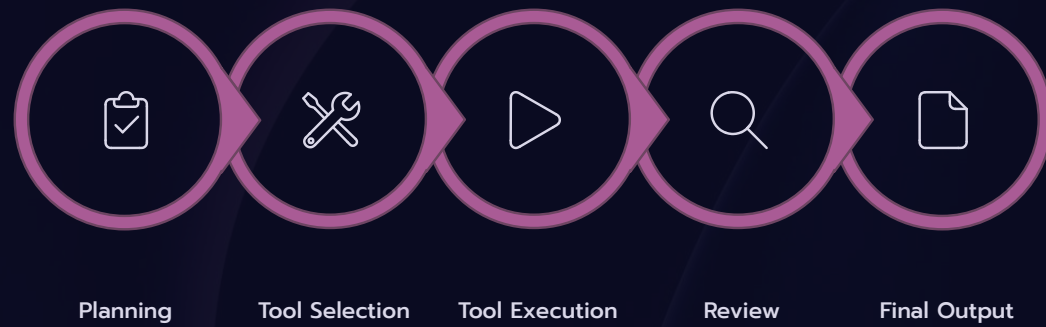
Evaluation Tests

Run the full evaluation dataset. Measure task success, policy compliance, and behavioral consistency across all representative scenarios.

Debugging Agent Failures

Debug the execution trace, not just the final answer

Locate the Failure in the Workflow



Useful Evidence to Collect

- Prompt version & model used
- Full context sent to the LLM
- Tool arguments & tool results
- Workflow state at each step
- Retry history & retry counts
- Reviewer decision & rationale

Failures in agentic systems are rarely simple. The root cause is often several steps removed from where the error surfaces. Trace backward through the execution chain.

Responsible AI

Enterprise agents can affect people and business decisions at scale



Human Oversight

Design explicit approval gates at consequential decision points. Autonomy should be earned through demonstrated reliability, not assumed from capability.



Data Protection

Agents that access sensitive data must enforce data-handling policies — not rely on the LLM to self-regulate what it shares or stores.



Auditability

Every consequential action must be traceable. Logs, traces, and approval records are the foundation of accountability in enterprise AI systems.

The greater the consequence of an agent's action, the stronger the controls around that action should be.



Production Readiness Checklist

Ask these questions before any deployment



Architecture

Is the workflow clearly defined? Are agent responsibilities explicit and bounded?



Reliability

Are failures handled? Are retries controlled? Is escalation defined?



Evaluation

Is there an evaluation dataset? Are success criteria defined? Has the agent been tested?



Security

Are secrets protected? Is authorization enforced at tool boundaries?



Observability

Are logs available? Can the workflow be traced? Are alerts configured?



Cost

Are token usage and LLM calls measured? Is unnecessary work minimized?

Deployment Considerations

The agent must fit into the enterprise environment — not the other way around

Deployment Checklist

- Application packaging & runtime environment
- Configuration management
- Secret management at runtime
- Network access controls
- Authentication & authorization integration
- Structured logging & monitoring hooks
- Horizontal scaling & rollback capability

Provider Flexibility

The training architecture supports both **Claude** and **Azure OpenAI** through a common LLM abstraction layer. Provider selection is driven by configuration — never by hard-coded agent logic.

This matters in enterprise environments where approved providers, model versions, or deployment policies may change after initial rollout. The agent should survive that change without a rewrite.

Model Independence

Separate agent engineering from model selection

Why Model Independence Matters

Tightly coupling an agent to a specific model creates fragility. Approved providers change, models are deprecated, and cost considerations shift over time. A well-engineered agent operates through a common LLM interface — the underlying model is a configuration decision, not a design constraint.

This is a critical distinction for enterprise environments:

Agent engineering ≠ model dependency.

Participants in this program learn to build agents that can potentially operate against any approved model — giving organizations the flexibility to adapt to evolving AI procurement and governance policies.

Common LLM Interface

The agent calls a single abstraction layer, not a provider-specific SDK directly.

Claude

Configured via environment and provider settings.

Azure OpenAI

Swapped in without changing agent logic.

The Production Agent Blueprint

Every capstone agent should answer these twelve questions

1 What problem does the agent solve?

Define the business or operational problem and why an agent is the right solution.

2 What is the architecture?

Describe the full system — components, interfaces, and enterprise control layers.

3 What tools can it use, and what can it access?

Enumerate tools, data sources, and action boundaries explicitly.

4 Where is human approval required?

Identify every consequential decision point that requires human review.

5 How are failures handled, evaluated, and monitored?

Define recovery paths, evaluation criteria, and operational monitoring strategy.

6 How is it secured, costed, and deployed?

Document security controls, cost levers, and the deployment model.

POC vs. Production

Productionization is the process of turning an impressive demo into a dependable enterprise capability

Dimension	POC	Production
Agent works	Demonstrated	Reliable & consistent
Tools	Demonstrated	Secured & authorized
Errors	Basic handling	Recovery + escalation
Secrets	Development setup	Managed securely
Testing	Few examples	Evaluation dataset
Logging	Basic	Structured observability
Monitoring	Limited	Operational monitoring
Cost	Estimated	Measured & controlled
Security	Demonstrated	Enforced deterministically
Deployment	Local	Enterprise environment
Governance	Informal	Defined controls

Your Production Agent

Take the agent from Lab 4 and add every production layer



Security

Authentication + Authorization + Tool access controls



Reliability

Retry logic + Input validation + Failure handling



Observability

Structured logging + Workflow tracing + Operational monitoring



Evaluation

Representative test dataset + Defined success metrics



Cost

Token optimization + LLM call minimization

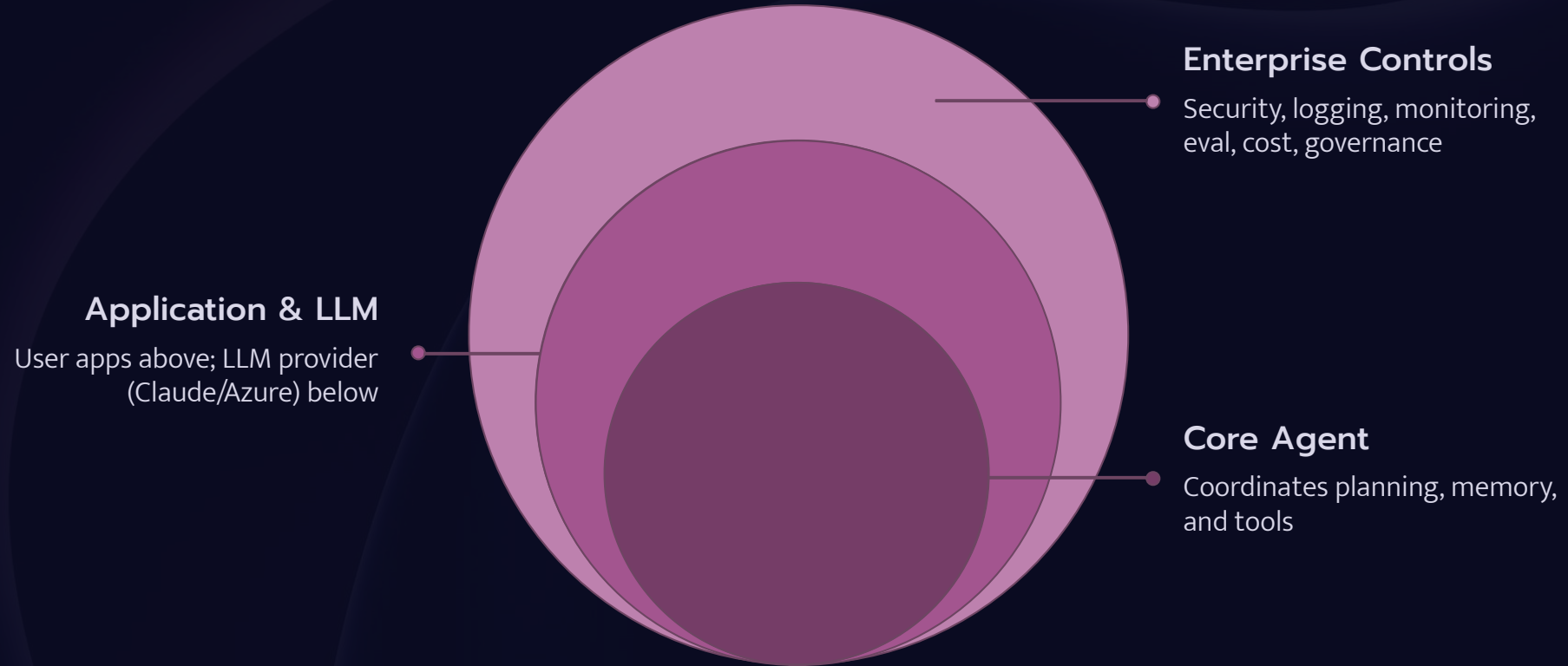


Governance

Human approval gates + Full auditability

Final Architecture

Every layer serves a purpose — none is optional in production



This is the complete picture. The agent core handles reasoning and orchestration. The LLM provider executes inference. The enterprise controls layer — spanning the entire stack — ensures that autonomy is bounded, observable, evaluable, and economically accountable. No single layer is sufficient without the others.

The Production Mindset

Don't ask only: "**Can the agent do it?**"

Ask: "**Can we trust it to do it repeatedly, safely, observably, and economically?**"

Useful

Solves a real problem at enterprise scale

Reliable

Handles failures and recovers gracefully

Secure

Enforces authorization deterministically

Observable

Every action is logged, traced, and monitored

Evaluated

Behavior is measured against defined criteria

Economically Viable

Costs are measured, controlled, and justified

Key Takeaways

Production-Ready Agent Engineering — the eleven principles

1 Configuration is part of architecture.

Externalize everything that varies by environment.

2 Prompts must be versioned.

A small change can shift agent behavior significantly.

3 Secrets must never live in source code.

Use environment variables and secret managers exclusively.

4 Authorization must be enforced outside the LLM.

Deterministic controls, not probabilistic reasoning, enforce policy.

5 Tools are security boundaries.

Control access at the tool layer, not at the prompt layer.

6 Failures require controlled recovery.

Define retry, escalation, and fallback paths explicitly.

1 Agent workflows need logs and traces.

Observability is a prerequisite for operation, not an afterthought.

2 AI behavior requires evaluation.

Traditional testing alone cannot cover the behavioral space of an agent.

3 Optimize the workflow, not just the prompt.

Token and call costs compound across every step in the chain.

4 Human approval belongs at consequential decision points.

Autonomy must be bounded by the stakes of the action.

5 Deployment requires operational controls.

Packaging, configuration, scaling, and rollback are not optional.

📌 **Final principle:** A production AI agent is not merely an LLM connected to tools. It is an engineered enterprise system with controls around autonomy.

The Complete Journey

Six decks — from first principles to production deployment



You have now built all the pieces. The next step is applying them to your capstone — a production agent blueprint that is **secure, observable, evaluated, and deployable**.